METASCALE HEALTH — PRODUCTION MASTER BLUEPRINT (v2.0)

Principal Architecture Review · Technical Execution Masterplan · Implementation Source-of-Truth
Version 2.0 — April 2026 | Updated from v1.0 with explicit gap fixes and clarified decisions.

---

0. Document Purpose and Delta from v1.0

This v2.0 blueprint supersedes v1.0 of the “METASCALE HEALTH — PRODUCTION MASTER BLUEPRINT”. It keeps the overall structure and intent of v1.0 but:

- Makes **implicit assumptions explicit**, especially around `.pkl` verification, liver-model usage policy, and risk thresholds.
- Fills the previously missing or weakly specified parts (model verification gate, confidence modes, doctor–patient access workflow, calibration plan, MVP vs full-production rollout).
- Tightens language so that no reader can accidentally treat the current SPA prototype as "already ML-integrated" or "production-safe".

Where there is any conflict, **this v2.0 document is the only source of truth**.

---

1. Executive Understanding of the Current Project (unchanged in essence)

Metascale Health is a single-page application (SPA) in vanilla HTML/CSS/JS for **anonymous screening of liver disease and diabetes risk for Indian users**, with a doctor-facing portal shell in the same page. The current live artefact is a **polished prototype only** — there is no real backend, no persistence, and all risk scores are heuristic. The goal is to evolve this into a production-ready, medically responsible screening & triage tool.

Key facts (as of April 2026):

- Frontend: single `index.html`, `styles.css`, `app.js` plus logo PNG.
- Patient flow: 6-step questionnaire → synthetic risk scores → dashboard with ring, badges, factors, and a prototype AI explanation placeholder.
- Doctor flow: in-page tabs → register/login stored in JS variable → mock dashboard, dummy patient list, modal detail view, prototype doctor AI summary placeholder.
- ML artefacts exist (cleaned CSVs, notebooks, `.pkl` models) but are **not wired to the UI at all**.

v1.0 captured this correctly; v2.0 only clarifies:

- We formally classify the SPA as **Prototype / Demo**. No part of it should be user-facing in a clinical or quasi-clinical context without the backend described later.
- Any commit that exposes the v1 UI must carry a visible "Prototype only — no real medical ML" banner until the full stack is implemented.

---

2. File-by-File Understanding (v2 clarifications only)

This section keeps the v1.0 file inventory but adds clarifications where there were hidden assumptions.

2.1 Frontend Files

- **index.html**
  - SPA shell, all sections pre-rendered.
  - Tabs/sections for: language splash, landing, role selection, user questionnaire, results dashboard, doctor auth, doctor dashboard.
  - Risk: all sections in DOM at once, including doctor area and any sensitive data — needs hardening once real data is present.
- **styles.css**
  - Saffron-gradient, glassmorphism, Playfair + DM Sans typography; overall visually coherent for Indian wellness context.
  - Media queries partially present; mobile responsiveness is decent but not formally tested for all devices.
  - **v2 requirement**: treat this file as a *visual spec* only; when moving to React or another framework, re-implement tokens (colors, radius, spacing) as design system primitives instead of copying raw CSS.
- **app.js**
  - Contains: navigation, validation, questionnaire state, feature mapping, heuristic risk simulator, mock doctor auth, dummy patient list, modal logic, and AI placeholder functions.
  - **Important mismatches (already identified in v1, kept here as hard requirements to fix):**
    - `FamilyDiabetes` vs `Family_Diabetes` model column.
    - `JunkFood` encoding 0–3 vs training 1–4.
    - `RegularMedicine` hardcoded to 0 and not collected from UI despite being top feature in diabetes model.
    - Liver features in UI are a strict subset of what the liver model needs; synthetic `synthetic_liver_lifestyle` does not exist in the trained model.
  - **v2 clarification**: this file must be treated as *legacy prototype code*. The production implementation will move mapping and risk computation to backend. Frontend role = form collection + rendering of API results only.

2.2 Data and ML Artefacts

- **diabetes_cleaned_dataset.csv**
  - 950 rows, 17 features + target.
  - Encoded according to rules in diabetes_dataset_logic.ipynb (age bins, JunkFood 1–4, etc.).
- **ilpd_cleaned.csv**
  - 583 rows, 10 features + target, heavily imbalanced (~71% disease).
  - Liver model expects *all lab fields* (total_bilirubin, direct_bilirubin, alkaline_phosphotase, alt, ast, total_proteins, albumin, albumin_globulin_ratio) plus age and gender.
- **Notebooks** (analysis_both.ipynb, diabetes_dataset_logic.ipynb, liver_dataset_cleaning.ipynb)
  - Canonical source of cleaning and encoding logic.
  - No scaler or OHE persisted into the .pkl models; models expect **pre-encoded numeric features only**.
- **Models (diabetes_model.pkl, liver_model.pkl)**
  - RandomForest-based classifiers saved via joblib.
  - v1 assumed feature_names_in_ order based on local inspection; **v2 adds an explicit verification gate (Section 6).**

---

3. Data / Feature Consistency Audit (v2 summary)

The v1 tables listing **diabetes** and **liver** feature contracts remain valid and should be reused as-is. v2 adds:

1. A strict rule that **UI → backend feature construction must happen only in backend**, not in JS, to avoid drift.
2. A **Runtime Contract Check**: at backend startup, assert that model.feature_names_in_ exactly matches the hard-coded expected list in feature_contracts.py. If not, the API refuses to start.
3. A **Training-Only Features Rule**: any new engineered features must be added *first* in the training notebook, then carried into

the cleaned CSV and model; they must never be invented only at inference time.

The concrete mismatches and missing fields listed in v1 (JunkFood, Family_Diabetes, RegularMedicine, liver labs) stay as **non-negotiable bugs** to be fixed in backend and, where needed, in the UI.

---

4. Gap Audit (v2 additions)

v1's gap tables are correct. v2 adds precise policies where v1 only suggested direction.

4.1 Added Product Policies

1. **Prototype Banner Policy**
   - Until the full backend described here is deployed and verified, the SPA must display an always-visible banner: "Prototype only — risk scores are dummy simulations, not real ML outputs."
   - This banner can be removed only after: backend, model integration, and calibration steps are complete and manually QA'd.
2. **Liver Prediction Availability Policy**
   - Define liver prediction modes (see Section 10):
     - `liver_mode="full-labs"` – all 8 lab fields provided.
     - `liver_mode="partial-labs"` – at least 4 core labs present; others imputed.
     - `liver_mode="no-labs"` – liver RF model **not called**; surface only qualitative lifestyle feedback.
   - The UI must only show a numeric liver risk when mode is `full-labs` or `partial-labs`, and it must show a confidence banner when `partial-labs` is used.
3. **Doctor–Patient Linking Policy**
   - A doctor can see a screening only if either:
     - The screening was created from within that doctor's session (e.g., clinic workflow), or
     - A patient explicitly shares a **session code** or **PDF report ID** with the doctor, and the backend validates that code.
   - There is no global "see all anonymous screenings" view.

4.2 Added ML and Calibration Gaps

New explicit gaps for v2:

- **Calibration not yet implemented** — we still do not know how calibrated the RF probabilities are on held-out data.
- **Risk band thresholds are arbitrary (0.33 / 0.66)** — these must be derived from ROC curves, precision–recall trade-offs, and discussion with a medical advisor.
- **Liver imbalance mitigation not used** — threshold tuning or class_weight adjustments are not yet part of the training pipeline.

These are resolved via the calibration plan in Section 10.

---

5. Locked Decisions / Source of Truth (v2 tightened)

The v1 locked decisions are mostly good; v2 clarifies and strengthens a few.

5.1 Product Scope (Locked, clarified)

- Patient-facing, anonymous (session-based) screening for **diabetes and liver risk** using tree-based classifiers.
- Doctor-facing, authenticated portal for **viewing screening histories, risk bands, trends, and AI-generated clinical notes**.
- **No diagnosis or prescription**; only risk stratification and suggested follow-up steps.
- AI explanation via Gemini (or compatible LLM) with separate prompt templates and output structures for **patient** and **doctor**.
- PDF report generation for patient download and doctor upload into EMR if needed.
- i18n-ready, English first; Hindi and possibly regional languages later.

5.2 Architecture Style (Locked, with migration plan)

- Backend: **FastAPI monolith** with the following layers:
  - routers/ (auth, screenings, doctor, ai, health)
  - services/ (preprocessing, prediction, pdf, ai)
  - ml/ (model registry + feature contracts)
  - database/ (SQLAlchemy models + Alembic migrations)
  - config/ (Pydantic settings).
- Frontend:
  - **Phase 1**: existing vanilla SPA, thinly wired to backend via fetch.
  - **Phase 2**: migration to React (or similar) with componentized UI and shared design tokens.
- Database: PostgreSQL as the single source-of-truth for screenings, predictions, AI explanations, auth, and audit logs.

5.3 MVP vs Full Production Tracks

To avoid paralysis, we lock a **two-track execution**:

- **Track A — MVP (Clinic demo / pilot)**
  - Full backend implemented (auth, screenings, ML, AI explanations, DB).
  - Existing SPA wired to backend.
  - Basic calibration + risk bands done, but not yet medically validated at scale.
  - Deployed to limited clinician partners only under explicit "pilot" label.
- **Track B — Production Launch**
  - React (or equivalent) frontend.
  - Hardened auth, rate-limiting, full observability, backup, retention policy enforced.
  - Calibration tuned with real-world feedback.
  - Legal/medical sign-off obtained.

---

6. Final System Architecture (v2 with Verification Gate)

The high-level architecture from v1 stays, but v2 adds a **Model Verification Gate** at startup.

6.1 Components

- **Web Frontend** (Phase 1: SPA; Phase 2: React app)
  - Serves questionnaire, results dashboard, doctor portal UI.
  - Calls backend via HTTPS using JSON; relies on cookies for auth.
- **API Backend (FastAPI)**
  - Exposes `/api/auth/*`, `/api/screenings/*`, `/api/doctor/*`, `/api/ai/*`, `/health`.
  - Holds model registry and preprocessing logic.
  - Enforces all validation, rate limiting, access control, and audit logging.
- **PostgreSQL Database**
  - Houses `doctors`, `patient_sessions`, `screenings`, `diabetes_predictions`, `liver_predictions`, `ai_explanations`, `refresh_tokens`, `audit_logs`.
- **ML Artifact Store**
  - Directory or bucket with `.pkl` models + `model_metadata.json`.
  - Versioned, with reference in DB rows.
- **LLM Provider (Gemini / Sarvam / OpenAI-compatible)**
  - Accessed via backend only; frontend never holds API keys.
  - Used for both patient-friendly and doctor-clinical summaries.

6.2 Model Verification Gate (New)

On app startup:

1. Load each `.pkl` model with joblib inside `model_registry.load_all()`.
2. Extract `feature_names_in_` and compare against the **hard-coded expected feature list** in `feature_contracts.py`.
3. For each model, run a **known test vector** (small JSON file in repo) through the preprocessing + prediction path and assert

that:
    - `predict_proba` returns a 2-element vector.
    - Probabilities sum to 1 (within tolerance).
    - Class indices align with `classes_` (0 = healthy, 1 = disease).
4. If any check fails, set `models_loaded = False` and refuse to serve `/api/screenings` (return 503 with JSON error).

---

7. Backend Architecture (v2 clarifications)

Directory structure (simplified):

```
backend/
  app/
    main.py
    config.py
    database.py
    ml/
      model_registry.py
      feature_contracts.py
      model_metadata.json
    services/
      preprocessing.py
      prediction_service.py
      ai_service.py
      pdf_service.py
    routers/
      auth.py
      screenings.py
      doctor.py
      ai.py
      health.py
    schemas/
      auth.py
      screening.py
      doctor.py
      ai.py
    middleware/
```

```
    auth_middleware.py
    rate_limit.py
```

Key decisions:

- Business logic (preprocessing, prediction, AI prompting) lives under `services/`, never inside routers.
- Preprocessing functions are **pure** (no DB access), tested independently.
- Prediction service never constructs feature vectors from raw request JSON directly; it always uses `feature_contracts` + preprocessing outputs.

---

8. Database Design (unchanged + clarified semantics)

The v1 schema for `doctors`, `patient_sessions`, `screenings`, `diabetes_predictions`, `liver_predictions`, `ai_explanations`, `refresh_tokens`, and `audit_logs` remains valid.

v2 clarifies:

- `screenings.raw_inputs` is an **immutable JSON snapshot** of the original questionnaire answers as sent from frontend.
- `*_predictions.feature_snapshot` is the **exact feature vector** used for model prediction, in the same order as `feature_names_in_`.
- `*_predictions.imputed_fields` explicitly records which fields were imputed and with what values.
- `ai_explanations` has `audience = 'patient' | 'doctor'`. Each pair `(screening_id, audience)` is unique.

---

9. Authentication + Security Design (v2 minor tightenings)

The v1 design (JWT + refresh tokens in HttpOnly cookies, CSRF token for state-changing requests, rate limiting on auth) is kept.

v2 adds:

1. **Device-bound refresh tokens (optional for Phase 2)** — include `user_agent` + partial IP hash in `refresh_tokens.metadata` and cross-check on use.
2. **Session header for anonymous patients** — a randomly generated UUID (X-Session-ID) created on first questionnaire visit, stored in `patient_sessions`.
3. **Doctor verification flag** — `doctors.is_verified` must be true for access to any patient-level data.

---

10. ML Integration Design (v2 with Calibration + Confidence)

10.1 Preprocessing (no changes in logic, but now canonical)

- Diabetes preprocessing:
  - Age binning, JunkFood encoding (1–4), SoundSleep derivation, etc., as per `diabetes_dataset_logic.ipynb`.
- Liver preprocessing:
  - Raw numeric labs (or median imputation), gender binary, etc., as per `liver_dataset_cleaning.ipynb`.

These are fully specified in code comments and unit tests; no hidden transformations.

10.2 Prediction Pipeline

For each `POST /api/screenings` request:

1. Validate JSON body against `ScreeningRequest` Pydantic schema.
2. Persist `patient_session` if not yet present (using X-Session-ID).
3. Store `raw_inputs` in `screenings`.
4. Run diabetes preprocessing → feature vector → diabetes model → probabilities + risk band.
5. Run liver preprocessing **only if** sufficient labs are present (see modes below).
6. Insert rows into `diabetes_predictions` and `liver_predictions` as appropriate.
7. Return combined response structure including `confidence_mode` and `explanation_status`.

10.3 Risk Bands and Confidence Levels (New, v2)

Each disease prediction now has:

- probability_disease (float 0–1).
- risk_level ('low' | 'medium' | 'high') determined from calibrated thresholds.
- confidence_mode with values:
  - "full-features" — no imputation, all features within training ranges.
  - "partial-imputation" — some features imputed, but enough signal exists.
  - "unsupported" — model not called; UI must not display numeric risk.

10.4 Calibration and Threshold Selection (New)

Before any production use:

1. For both models, run the held-out validation set to obtain predicted probabilities.
2. Plot reliability curves; evaluate Brier score.
3. If miscalibrated, fit a simple calibration layer (e.g., Platt scaling or isotonic) and store coefficients separately (do **not** bake them into .pkl to keep inspection easy).
4. Choose RISK_THRESHOLD_LOW and RISK_THRESHOLD_HIGH to achieve a clinically acceptable balance between sensitivity and specificity, with preference for **higher sensitivity** (catch more true positives) at the cost of more mild false positives.
5. Save these thresholds and calibration parameters into model_metadata.json and environment config.

The concrete numeric thresholds are **not hard-coded in this PDF**; they must be derived empirically.

10.5 Liver Usage Modes (Formalised)

The backend decides one of three modes based on labs present:

- **Mode A — full-labs**
  - All 8 liver lab fields provided.
  - Model used normally; confidence_mode=full-features.

- **Mode B — partial-labs**
  - At least `age`, `gender`, and any **4 or more** of the 8 lab fields provided.
  - Missing labs filled with training medians; `confidence_mode=partial-imputation`.
  - UI must render an info banner like: “Some liver lab values were missing, so this estimate is less certain.”
- **Mode C — no-labs**
  - Fewer than 4 lab fields; RF model **not called**.
  - Liver card shows qualitative lifestyle hints only (no numeric percentage, no ring segment).
  - This is a hard safety rule.

---

11. AI Explanation Architecture (v2 with Output Schema)

The v1 design sketched Gemini usage. v2 locks:

11.1 Output Schema

For **patients**, the LLM output must be parsed into:

```
{
  "summary": "string",
  "what_it_means": "string",
  "key_factors": ["string"],
  "suggested_next_steps": ["string"],
  "safety_disclaimer": "string"
}
```

For **doctors**, the output schema is:

```
{
  "soap_subjective": "string",
  "soap_objective": "string",
  "soap_assessment": "string",
  "soap_plan": "string",
```

```
  "red_flags": ["string"],
  "follow_up_tests": ["string"]
}
```

The backend must:

- Pass a **strict system prompt** that forbids diagnosis, prescriptions, and guarantees that it will always include the safety disclaimer.
- Validate that all required fields are present; otherwise, mark explanation as failed and show a fallback message.

11.2 Caching and Rate Limiting

- Explanations are generated **per screening**, not per request — once generated, they are stored in `ai_explanations` and reused.
- Rate limit: no more than N explanations per session per hour (configurable).
- If rate-limited, UI shows a “try later” banner instead of error.

---

12. Frontend Architecture (v2 with Migration Plan)

Phase 1 (current SPA + backend wiring):

- Use existing HTML/CSS/JS.
- Replace `simulateRiskScores` with a `fetch('/api/screenings', ...)` call.
- Replace AI placeholder functions with calls to `/api/ai/explain/{screening_id}`.
- Add a persistent prototype/production status banner.

Phase 2 (React or similar):

- Componentise questionnaire, dashboard, and doctor portal.
- Move design tokens into a central theme file.
- Implement proper routing, state management, and accessibility from scratch.

---

13. Detailed Screen / Page Plan (unchanged with minor copy edits)

Reuse v1 list of screens (patient questionnaire steps, results dashboard sub-panels, doctor portal tabs), but ensure that:

- Every place that shows a risk number also shows **risk band** + **confidence label**.
- History views clearly show whether older screenings used prototype heuristics or real models (flag old ones).

---

14. End-to-End Implementation Plan (v2 with Two Tracks)

The v1 step list (model verification → env setup → backend skeleton → DB → auth → preprocessing tests → ML integration → persistence → AI → PDF → frontend wiring → hardening) is valid. v2 adds track labels:

- Steps 1–10 = **MVP track**.
- Steps 11+ (React migration, device-bound refresh tokens, full observability, advanced calibration) = **Production track**.

---

15. File-by-File Implementation Blueprint (unchanged with new tags)

Keep v1, but mark each item with `MVP` or `PROD` tags so you know what is required for a pilot vs full-scale launch.

---

16. API Contracts (v2: confidence + modes)

The endpoints from v1 remain the same. v2 only updates the **response schema** for `/api/screenings` to include:

```
{
  "screening_id": "uuid",
  "created_at": "ISO8601",
```

```
  "diabetes": {
    "probability": 0.63,
    "risk_level": "medium",
    "confidence_mode": "full-features"
  },
  "liver": {
    "mode": "full-labs",  // or partial-labs, no-labs
    "probability": 0.48,
    "risk_level": "medium",
    "confidence_mode": "partial-imputation"
  },
  "ai_explanation_status": "pending|ready|failed"
}
```

---

17. Validation Rules (v2 clarifications)

Add explicit rules for:

- Max pregnancies capped at 4 in preprocessing (values >4 are clipped and logged).
- Reject requests where obviously inconsistent fields appear (e.g., negative age, BMI < 10 or > 60).
- Log but accept rare codes mapped into nearest valid training category (e.g., JunkFood "rarely" → 1).

---

18. Testing Strategy (v2 essentials)

- **Unit tests** for preprocessing (each feature, edge cases, out-of-range).
- **Integration tests** that run a synthetic but realistic request through `/api/screenings` and compare outputs to expected probabilities (within tolerance).
- **Security tests** for auth flows, CSRF, and rate-limiting.
- **Snapshot tests** for AI explanations to ensure schema compliance (not content).

---

19. Observability + Operations (unchanged with explicit ML audit requirement)

All predictions must be logged in `audit_logs` with: model name, version, probability, risk level, confidence_mode, and anonymised session identifier.

---

20. Risks, Unknowns, and Blockers (v2 updated)

Newly emphasised risks:

- Liver predictions based on heavy imputation may be misleading if not clearly flagged; hence the new mode + confidence rules.
- Calibration and threshold choice are still pending actual analysis — cannot be hand-waved.
- Model artefacts could drift from notebooks if retrained without updating `feature_contracts.py` and metadata; mitigated by startup gate.

---

21. Final Execution Roadmap (v2)

Follow v1 step ordering, with these additional hard gates:

1. **Gate 0 — Model verification script passing** before any backend work branches off.
2. **Gate 1 — Preprocessing tests green** before hitting real models from API.
3. **Gate 2 — Calibration notebook finished and thresholds stored** before exposing risk bands to any real users.
4. **Gate 3 — Legal/medical review and consent copy approved** before a public or pilot launch.

Once all four gates are passed, you can consider the system a **properly engineered MVP** ready for controlled clinical pilot.